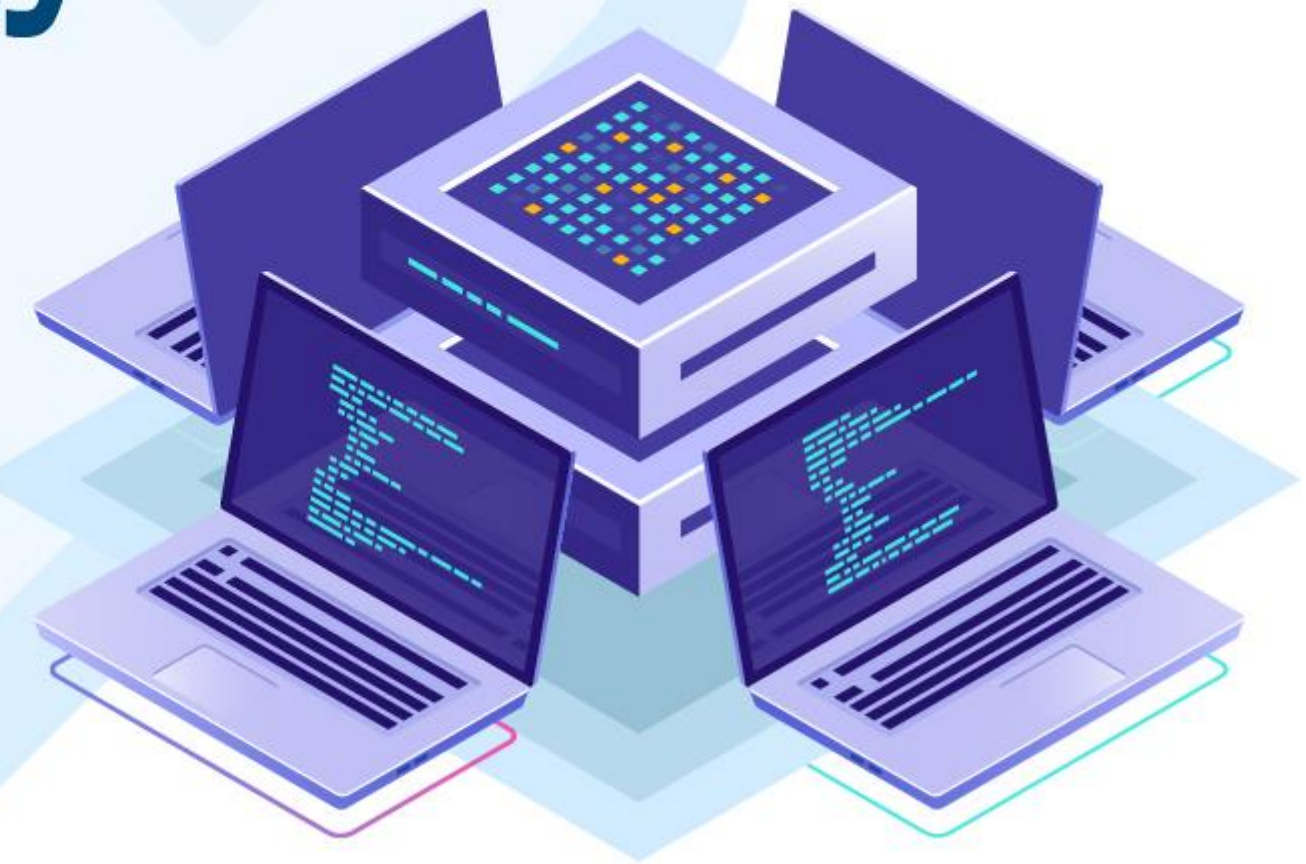
Information Technology

Year: 2026-2027

Fall Semester

- Computer Graphics
- **Dr. Yasmine Mahmoud**

Unit 2: Line drawing algorithms



EELU

الجامعة المصرية للتعلم الإلكتروني الأهلية

THE EGYPTIAN E-LEARNING UNIVERSITY

Line Drawing Algorithms

* Digital Differential Analyzer (DDA) (3rd Technique)

-The **DDA** method tries to **overcome** the **slowness** that exists in the **previous two** (methods). as they were using floating points and (+ , * , / , -) operations.

- DDA also use floating points but less than the previous two methods.

- The **DDA** (Digital Differential Analyzer) appeared in **1940** as **hardware** and **software**. And there are devices called **DDA** that can **scan** (or **render**) any line, as they rely on the **incremental method** to be able to scan any line (the incremental difference between the x-coordinates and y-coordinates of the two endpoints to plot the line).

- DDA is an **easy way** and a **faster** method compared to the previous two methods.

Line Drawing Algorithms

DDA Algorithm Steps:

The steps involved in DDA line generation algorithm are:

1. Input the two endpoints of the line segment, (x1,y1) and (x2,y2).
2. Calculate the difference between the x-coordinates and y-coordinates of the endpoints as Δx **and** Δy respectively.
3. Calculate the **slope** of the line as $m = \Delta y / \Delta x$.
4. Set the initial point of the line as (x1,y1).
5. Loop through the **x**-coordinates of the line, **incrementing by one** each time, and **calculate the corresponding y-coordinate** using the equation $y = y1 + m(x - x1)$.
6. Plot the pixel at the calculated (x,y) coordinate.
7. **Repeat** steps 5 and 6 until the endpoint (x2,y2) is reached.

EELU

Line Drawing Algorithms

DDA Algorithm Example:

- Draw a Line from (10 , 14) to (20 , 18)

Sol (First Way):

1. $\Delta x = x2 - x1 = 20 - 10 = 10$
2. $\Delta y = y2 - y1 = 18 - 14 = 4$
3. Calculate no. of steps,

   If $\Delta x \leq \Delta y$ then no. of steps = Δy

   If $\Delta x > \Delta y$ then no. of steps = Δx

Therefore, no. of steps is no. of iterations = max (Δx , Δy)

now, $\Delta x > \Delta y$ **then**, no. of steps = $\Delta x = 10$

4. Calculate x increment and y increment

x inc = Δx / no. of steps , yinc = Δy / no. of steps

xinc = 10 / 10 =1

yinc = 4/10 = 0.4

5. start the table >>>>>

iter	x	y	Round(x)	Round (y)
0	10(first x)	14 (first y)	10	14
1	10 + xinc = 10 +1 = 11	14 + yinc = 14 + 0.4 = 14.4	11	14
2	11 + 1 = 12	14.4 + 0.4 = 14.8	12	15
3	13	15.2	13	15
4	14	15.6	14	16
5	15	16	15	16
6	16	16.4	16	16
7	17	16.8	17	17
8	18	17.2	18	17
9	19	17.6	19	18
10	20 (last x)	18(last y)	20	18

EELU

Line Drawing Algorithms

DDA Algorithm Example:

- Draw a Line from (10 , 14) to (20 , 18)

Sol (2nd Way):

1. Calculate slope (m), $\mathbf{m} = \frac{y_2 - y_1}{x_2 - x_1}$ = (18-14) / (20 - 10) = **0.4**

2 . Calculate no. of steps,

If $\Delta x \leq \Delta y$ then no. of steps = Δy

If $\Delta x > \Delta y$ then no. of steps = Δx

$\Delta \mathbf{x} = \mathbf{4}$, $\Delta \boldsymbol{y} = \mathbf{0.4}$

3. Make the table (x , y)

If $m < 1$ then $x++$ & $y = y + m$

If $m = 1$ then $x++$ & $y++$

If $m > 1$ then $x = x + \frac{1}{m}$ & $y = y++$

now , **m < 1 then x + + & y = y + m**

iter	x	y	Round(x)	Round (y)
0	10(first x)	14 (first y)	10	14
1	10 + xinc = 10 +1 = 11	14 + **m** = 14 + 0.4 = 14.4	11	14
2	11 + 1 = 12	14.4 + 0.4 = 14.8	12	15
3	13	15.2	13	15
4	14	15.6	14	16
5	15	16	15	16
6	16	16.4	16	16
7	17	16.8	17	17
8	18	17.2	18	17
9	19	17.6	19	18
10	20 (last x)	18(last y)	20	18

Line Drawing Algorithms

DDA Algorithm:

- Advantages & Dis-advantages of DDA:

- Advantages:

1. It is the **simplest algorithm**, and it does not require special skills for implementation.
2. It is a **faster method** for calculating pixel **positions than the direct use of equation y = mx + b (Slope Intercept Formula)**.
3. It eliminates the multiplication in the equation by making use of raster characteristics, so that **appropriate increments** are applied in the x or y direction to find the pixel positions along the line path.

- Disadvantages:

1. **Floating point** arithmetic in DDA algorithm is still **time-consuming (Slow)**.
2. The algorithm is orientation dependent. So the end point accuracy is poor.

Digital Differential Analyzer (DDA) line Algorithm

Example 2: trace the line where the end points between (23,33), (29,40) by DDA method.

Solution:

$\Delta x = 29-23=6$

$\Delta y = 40-33=7$

Length=7

Xinc=6/7 =0.857

Yinc=7/7=1.

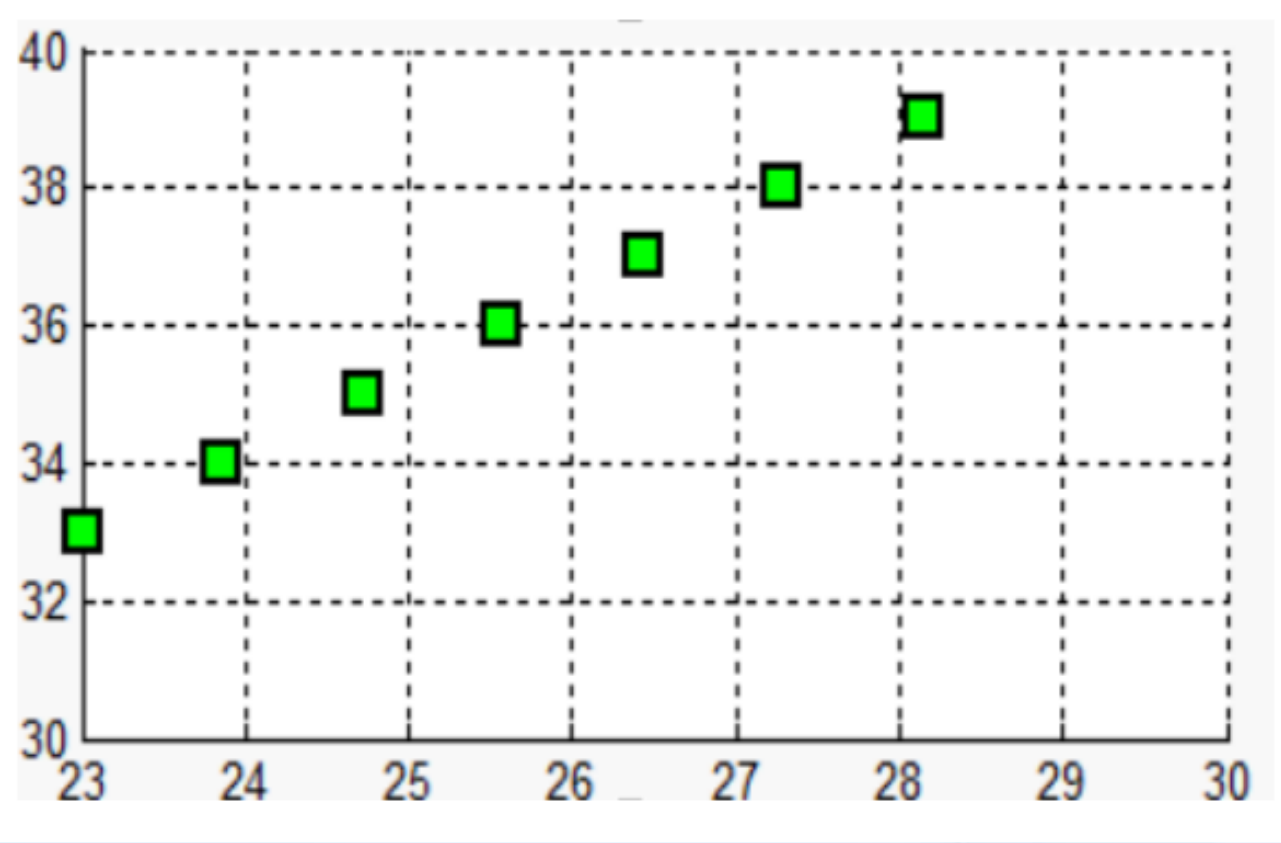
Using : **Round** is a function approximates float (real) numbers to nearest larger number.

i	*X*	*Y*	*Plot (X)*	*Plot (y)*
0	*23*	*33*	*23*	*33*
1	*23.857*	*34*	*24*	*34*
2	*24.714*	*35*	*25*	*35*
3	*25.571*	*36*	*26*	*36*
4	*26.429*	*37*	*26*	*37*
5	*27.286*	*38*	*27*	*38*
6	*28.143*	*39*	*28*	*39*
7	*29*	*40*	*29*	*40*

Digital Differential Analyzer (DDA) line Algorithm

Example 2: trace the line where the end points between (23,33), (29,40) by DDA method.

Solution:



i	*X*	*Y*	*Plot (X)*	*Plot (y)*
0	*23*	*33*	*23*	*33*
1	*23.857*	*34*	*24*	*34*
2	*24.714*	*35*	*25*	*35*
3	*25.571*	*36*	*26*	*36*
4	*26.429*	*37*	*26*	*37*
5	*27.286*	*38*	*27*	*38*
6	*28.143*	*39*	*28*	*39*
7	*29*	*40*	*29*	*40*

Digital Differential Analyzer (DDA) line Algorithm

Example 3 :

Consider the line from (0,0) to (6,6).

Use the simple DDA algorithm to rasterize this line

Solution:

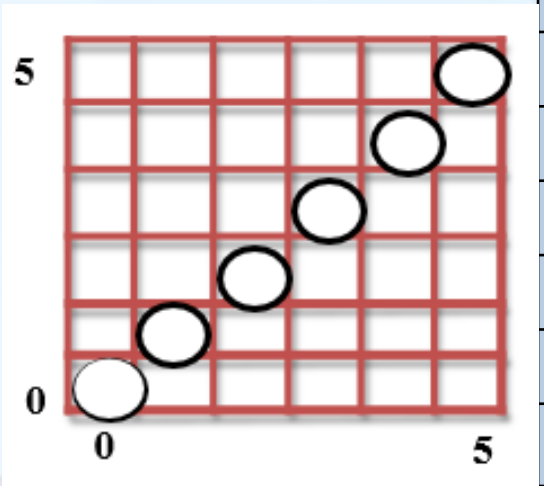
x1=0 , y1=0, x2=6 , y2=6

Length = $y2 - y1 = x2 - x1 = 6$

Xinc= $(x2 - x1)/length = 6/6 = 1$

Yinc= $(y2 - y1)/length = 6/6 = 1$

The result are plotted as shown



i	plot	x	y
		0	0
1	(0,0)		
		1	1
2	(1,1)		
		2	2
3	(2,2)		
		3	3
4	(3,3)		
		4	4
5	(4,4)		
		5	5
6	(5,5)		

Digital Differential Analyzer (DDA) line Algorithm

Example 3 :

Consider the line from (0,0) to (4,6).

Use the simple DDA algorithm to rasterize this line

Solution:

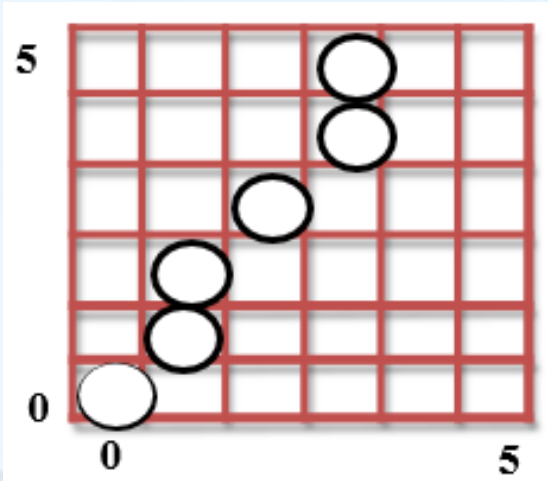
x1=0 , y1=0 , x2=4 , y2=6

Length = $y2 - y1 = 6$

Xinc= $(x2 - x1)/length = 4/6$

Yinc= $(y2 - y1)/length = 6/6 = 1$

The result are plotted as shown



i	plot	x	y
		0	0
1	(0,0)		
		0.66	1
2	(1,1)		
		1.33	2
3	(1,2)		
		2	3
4	(2,3)		
		2.66	4
5	(3,4)		
		3.33	5
6	(3,5)		

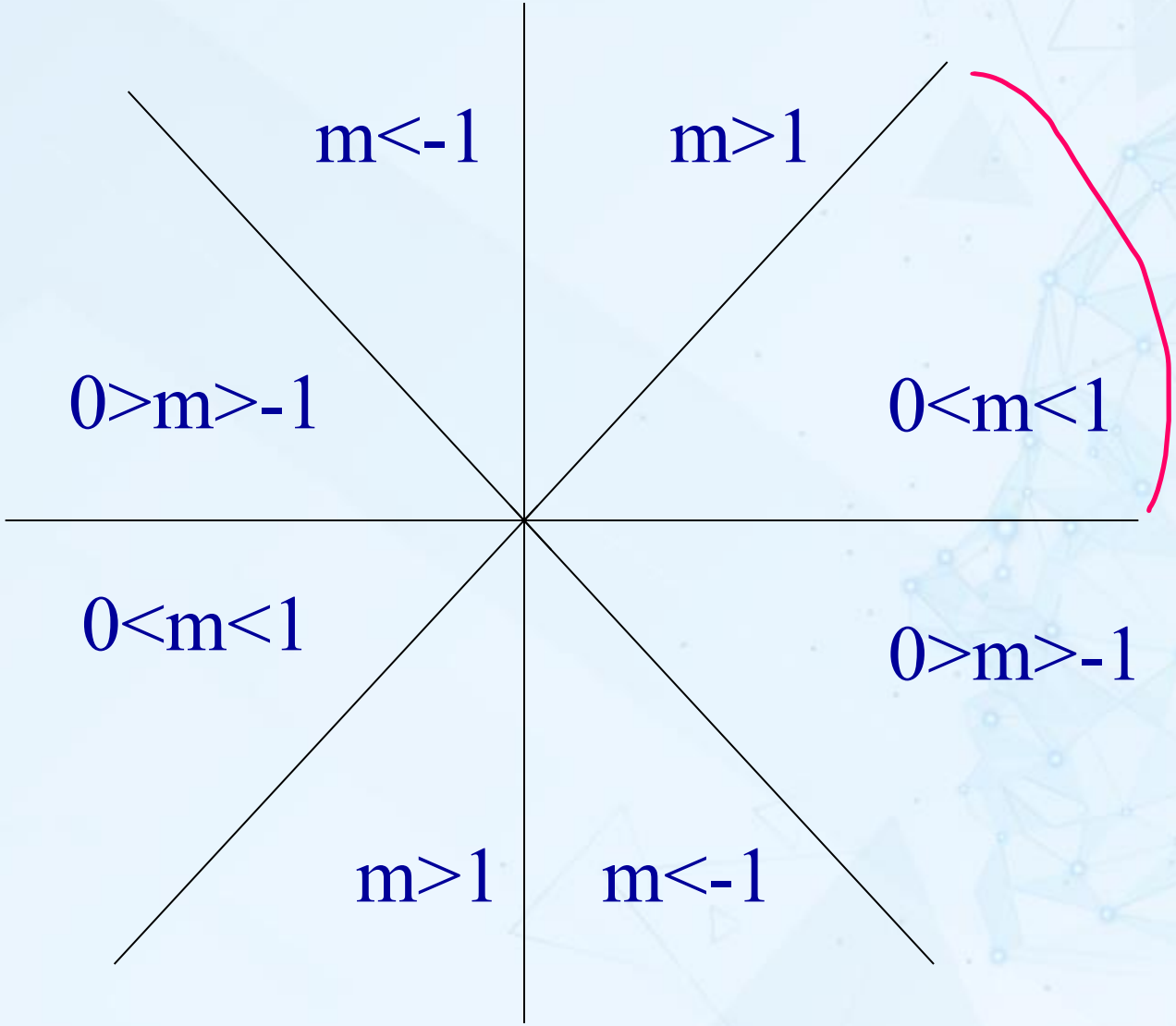
Bresenham’s Line Drawing Algorithm

- The basic principle of Bresenham's line algorithm is to select the optimum raster locations to represent a straight line.
- To accomplish this algorithm always increments either x or y by one unit depending on the slope of line.

- Accurate and efficient
- Uses only incremental integer calculations

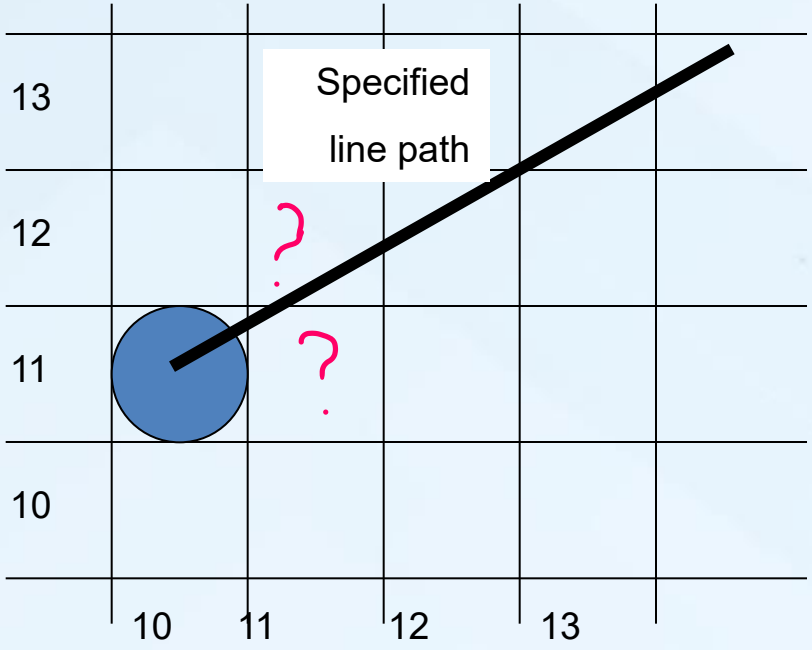
EELU

Line-drawing algorithm should work in every octant, and special cases



The method is described for a line segment with a **positive slope less than one**

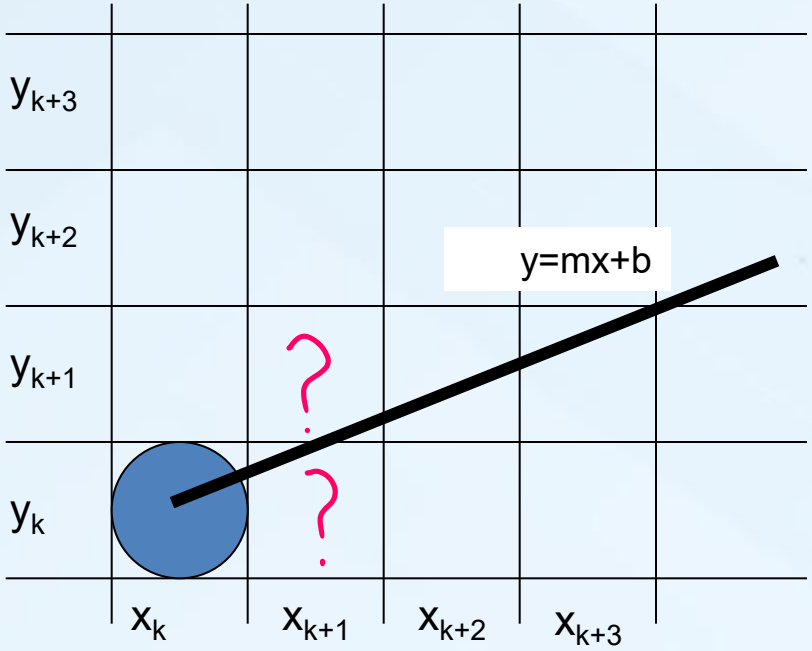
Bresenham's Line Algorithm



Decide what is the next pixel position (11,11) or (11,12)

Illustrating Bresenham's Approach





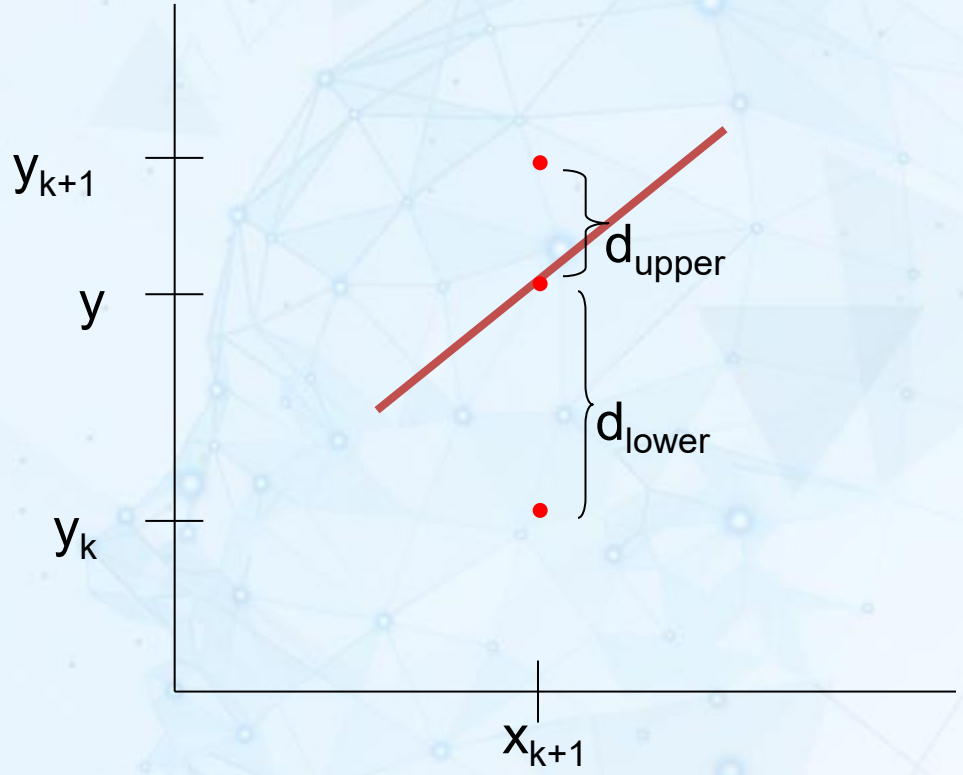
Because **m <1**
Then x++ (as the line will be more horizontally than vertically)

For the pixel position $x_{k+1}=x_k+1$, **which one we should choose?**
$(\mathbf{x_{k+1}},y_k)$ or $(\mathbf{x_{k+1}}, y_{k+1})$??

EELU

Bresenham's Approach

- $y=m(x_k + 1)+b$
- $d_{lower}=y-y_k=m(x_k + 1)+b-y_k$
- $d_{upper}=(y_k+1)-y= y_k+1 -m(x_k + 1)-b$



- $d_{lower}- d_{upper}= 2m(x_k + 1)+2y_k+2b-1$
- Rearrange it to have integer calculations:

  $m=\Delta y/\Delta x$

  Decision parameter: $p_k= \Delta x(d_{lower}- d_{upper})=2\Delta y.x_k - 2\Delta x. y_k + c$

The Decision Parameter

Decision parameter: $p_k = \Delta x(d_{lower} - d_{upper}) = 2\Delta y.x_k - 2\Delta x.\ y_k + c$

- p_k has the same sign with $d_{lower} - d_{upper}$ since $\Delta x > 0$.
- c is constant and has the value $c = 2\Delta y + \Delta x(2b-1)$
  - c is independent of the pixel positions and is eliminated from decision parameter p_k.

- If $d_{lower} < d_{upper}$ then p_k is negative.
  - Plot the lower pixel (East)
- Otherwise
  - Plot the upper pixel (North East)

Succesive decision parameter

- At step k+1

  $p_{k+1}= 2\Delta y.x_{k+1} - 2\Delta x. y_{k+1} + c$

- Subtracting two subsequent decision parameters yields:

  $p_{k+1}-p_k= 2\Delta y.(x_{k+1}-x_k) - 2\Delta x. (y_{k+1}-y_k)$

- $x_{k+1}=x_k+1$ so

  $p_{k+1}= p_k + 2\Delta y - 2\Delta x. (y_{k+1}-y_k)$

  – $y_{k+1}-y_k$ is either 0 or 1 depending on the sign of p_k
- First parameter p_0

  – $p_0=2\ \Delta y - \Delta x$

EELU

EELU

الجامعة المصرية للتعلم الإلكتروني الأهلية

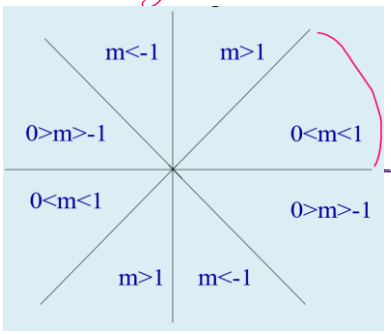
THE EGYPTIAN E-LEARNING UNIVERSITY

THANK YOU FOR WATCHING

QUESTIONS?



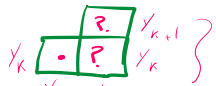
$P_0 = 2dy - dx$



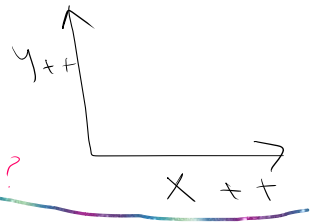
Mathematical Proof

(x_2, y_2)

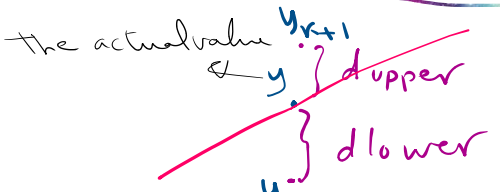
(x_1, y_1)



which pixel to choose?



- The current point (x_k, y_k)
- the next point $\begin{cases} (x_{k+1}, y_{k+1}) \\ \text{or} \\ (x_{k+1}, y_k) \end{cases}$



- if $d_{lower} < d_{upper}$ (closer to dlower)
  then the next point is (x_{k+1}, y_k)
- if $d_{upper} < d_{lower}$ (closer to dupper)
  the the next point is (x_{k+1}, y_{k+1})

Math. proof

Stright line equation

$$y = (m * x) + b \rightarrow (1)$$

- when pixel $\rightarrow x = x_{k+1}$ (next point)

$$\therefore y = m(x_k + 1) + b$$

- dlower: space between actual y value & the lower pixel.

- dupper: space between actual y value & the upper pixel.

- $d_{lower} = y - y_k$
- $d_{upper} = (y_k + 1) - y$

$$\therefore d_{lower} = m(x_k + 1) + b - y_k$$

& $$d_{upper} = (y_k + 1) - m(x_k + 1) - b$$

⇒ Calculate dlower - dupper

- if (dlower - dupper) > zero
  then dlower is bigger
- if (dlower - dupper) < zero
  then dupper is bigger

(*) dlower - dupper =

$= 2m(x_k + 1) + 2b - 2y_k - 1$ → (2)

↳ m in this equation is the only floating no.

$\therefore m = \frac{\Delta y}{\Delta x}$

∴ multiply equation 2 * $\boxed{\Delta x}$

$$\Delta x \; [d_{lower} - d_{upper}]$$

$$= \Delta x \left[2\underbrace{m}_{\frac{\Delta y}{\Delta x}}(x_k + 1) + 2b - 2y_k - 1 \right]$$

is P_k ⇒ Decision parameter

$$P_k = 2\Delta y (x_k + 1) + 2b\Delta x - 2\Delta x y_k - \Delta x$$

$$P_k = 2\Delta y x_k + 2\Delta y + 2b\Delta x - 2\Delta x y_k - \Delta x$$

Constant values

$$P_k = 2\Delta y x_k - 2\Delta x y_k + c$$

$$c = 2\Delta y + 2b\Delta x - \Delta x$$

$$\therefore P_{k+1} = 2\Delta y x_{k+1} - 2\Delta x y_{k+1} + c$$

$$\therefore P_{k+1} - P_k = 2\Delta y x_{k+1} - 2\Delta x y_{k+1} + c$$

$$= -2\Delta y\, x_k + 2\Delta x\, y_k - c$$

$$= 2\Delta y\,[x_{k+1} - x_k] - 2\Delta x\,[y_{k+1} - y_k]$$

if $x_k \rightarrow$ is x_5 index

$x_{k+1} \rightarrow$ is x_6

$x++$

$\therefore$ if $x_5 = 4$ so $x_6 = 5$

$\therefore x_{k+1} - x_k = 1$

$$p_{k+1} = \begin{cases} p_k + 2\Delta y \\ \text{or} \\ p_k + 2\Delta y - 2\Delta x \end{cases}$$